

Endpoints, DTO y pruebas de conexión

Actividad autónoma N.º 07 - Sesión 7

Estudiante: Roberto Samuel Valencia Saavedra

Asignatura: Desarrollo de Aplicaciones Móviles

Fecha de pruebas: 29 de setiembre de 2026

Plataforma verificada: Android Emulator, API 37.1

Proyecto: PharmaMobil (Kotlin Multiplatform / Compose / Ktor) y backend PharmaSoft (Spring Boot / Oracle). La evidencia documenta el comportamiento observado; no sustituye un resultado parcial por uno supuesto.

Repositorio Android

github.com/Akatosh4019/PharmaMobile_Valencia

Repositorio backend

github.com/Akatosh4019/BackendPharmobile

Alcance y criterio de evidencia

Se documentan cinco escenarios ejecutados en Android. El docente dispensó la evidencia de iOS; por ello no se declara una ejecución en esa plataforma. Los cambios temporales para provocar 404, timeout y JSON desconocido se revirtieron al finalizar.

Escenario	Evidencia observada
GET exitoso	GET /productos y /categorias a HTTP 200; lista visible.
ID inexistente	GET /productos/999999 a HTTP 404; error capturado, mensaje técnico.
Sin conexión	Modo avión a error de conexión; reintento posterior correcto.
Timeout	requestTimeoutMillis = 1 a vencimiento; app sigue abierta.
JSON desconocido	ignoreUnknownKeys = false a JsonConvertException; con true funciona.

1. Catálogo de endpoints

API base: **http://10.0.2.2:8080/api/v1/** desde Android Emulator; **http://localhost:8080/api/v1/** desde el PC. Versión: **v1**. Los códigos describen el contrato y las pruebas efectuadas; 401 no se declara porque esta API local no usa autenticación en estas rutas.

Método / ruta	Parámetros	Respuesta	Errores relevantes
GET /productos	pagina, tamaño (query; 0, 20 por defecto)	200 - PaginaProductosDto	400 en parámetros inválidos; 500 servidor
GET /productos/{id}	id (ruta)	200 - ProductoDto	404 inexistente (probado); 400 id inválido
POST /productos	ProductoRequestDto (JSON)	201 - ProductoDto, ID generado	400 validación/categoría inválida
PUT /productos/{id}	id + ProductoRequestDto	200 - ProductoDto	400 validación; 404 inexistente
DELETE /productos/{id}	id (ruta)	204 - sin cuerpo; baja lógica	404 inexistente
GET /categorias	Sin parámetros	200 - lista de CategoriaDto	500 servidor

El cliente de Android consume GET /productos, GET /categorias, POST, PUT y DELETE. GET /productos/{id} existe en el backend y se ejecutó para la prueba controlada de 404 mediante una modificación temporal de la URL en ProductoApi; no hay botón de búsqueda por ID en la interfaz final.

DELETE no elimina físicamente el registro: el backend marca estado = false. La app lo muestra en la pestaña Inactivos tras volver a consultar la lista. El contrato real devuelve 204, no 200.

2. Diccionario de DTO

Las tablas reproducen la nulabilidad y los valores por defecto declarados en Kotlin. «Sí» indica un argumento requerido para construir el DTO; los campos con valor predeterminado no lo son.

ProductoDto a Producto

JSON	Tipo Kotlin	Oblig.	Defecto	Dominio
id	Long	Sí	-	Producto.id
nombre	String	Sí	-	Producto.nombre
precio	Double	Sí	-	Producto.precio
stock	Int	Sí	-	Producto.stock
estado	Boolean	No	true	Producto.activo
categoriald	Long?	No	null	Producto.categoriald
categoriaNombre	String?	No	null	Producto.categoriaNombre

PaginaProductosDto a colección de Producto

JSON	Tipo Kotlin	Oblig.	Defecto	Uso
contenido	List<ProductoDto>	Sí	-	Se mapea a List<Producto>
pagina	Int	Sí	-	Paginación; no pasa al modelo
tamano	Int	Sí	-	Paginación; no pasa al modelo
totalElementos	Long	Sí	-	Metadato; no pasa al modelo
totalPaginas	Int	Sí	-	Metadato; no pasa al modelo
ultima	Boolean	Sí	-	Controla el bucle de carga

2. Diccionario de DTO (continuación)

ProductoRequestDto a datos enviados

JSON	Tipo Kotlin	Oblig.	Defecto	Origen en dominio
nombre	String	Sí	-	Producto.nombre
precio	Double	Sí	-	Producto.precio
stock	Int	Sí	-	Producto.stock
estado	Boolean	Sí	-	Producto.activo
categoriald	Long	Sí	-	Producto.categoriald (no nulo)

CategoriaDto a Categoria

JSON	Tipo Kotlin	Oblig.	Defecto	Dominio
id	Long	Sí	-	Categoria.id
nombre	String	Sí	-	Categoria.nombre

Fragmento JSON real del backend (GET /productos)

```
{
  "contenido": [{
    "id": 1, "nombre": "Paracetamol", "precio": 15.5,
    "stock": 100, "estado": true, "categoriaId": 1,
    "categoriaNombre": "Medicamentos",
    "fechaCreacion": "2026-09-29T20:04:30.325073"
  }],
  "pagina": 0, "tamaño": 20, "totalElementos": 7,
  "totalPaginas": 1, "ultima": true
}
```

Este fragmento abrevia el arreglo de siete productos, pero conserva los nombres y tipos devueltos. fechaCreacion es un campo adicional del backend; ignoreUnknownKeys = true permite omitirlo sin romper ProductoDto.

Código DTO correspondiente

```
@Serializable data class ProductoDto(
    val id: Long, val nombre: String, val precio: Double,
    val stock: Int, val estado: Boolean = true,
    val categoriaId: Long? = null,
    val categoriaNombre: String? = null,
)
```

3.1 Prueba 01 - Respuesta exitosa

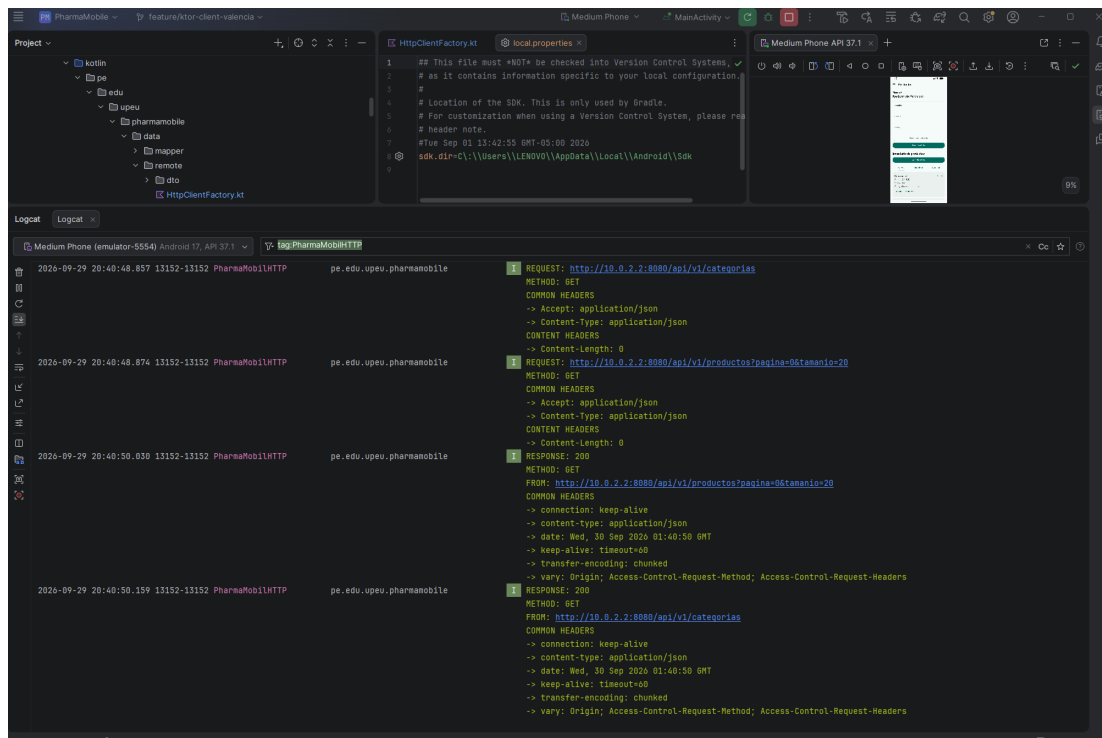
Fecha / plataforma: 29/09/2026, 20:40; Android Emulator API 37.1.

Pasos: Con Docker y backend encendidos, abrir la app y pulsar Actualizar lista. Filtrar Logcat por tag:PharmaMobilHTTP.

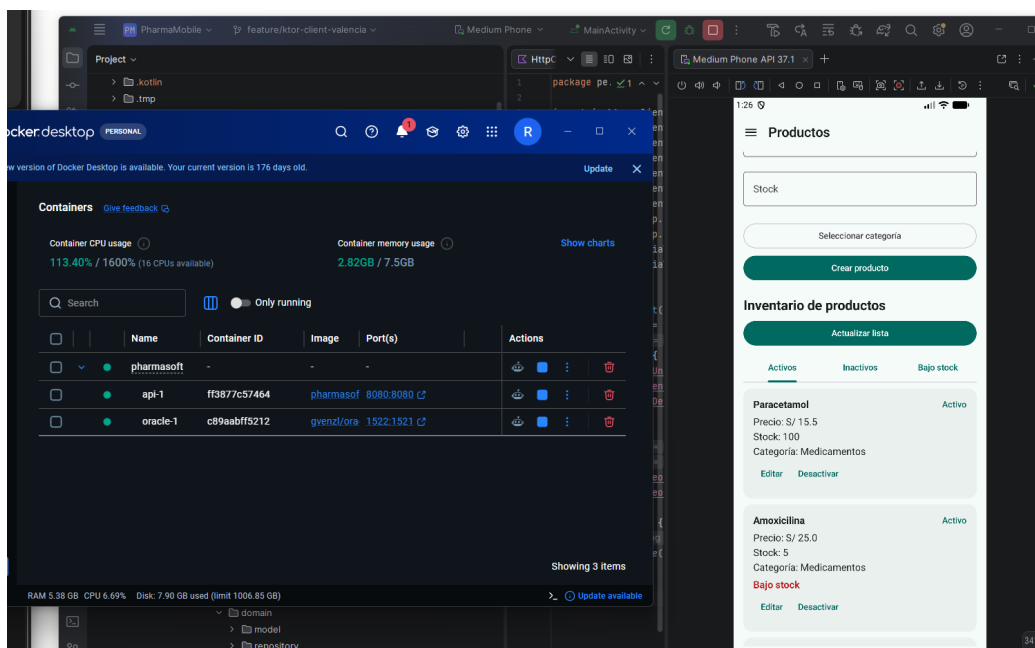
Esperado: HTTP 200 y lista de productos renderizada.

Observado: GET /api/v1/productos?pagina=0&tamano=20 a 200; GET /categorias a 200. Se muestran productos con nombre, precio, stock y categoría.

Conclusión: Conexión y deserialización normales verificadas.



Evidencia 01A. Petición y respuesta 200 en Logcat.



Evidencia 01B. Lista cargada en Android.

3.2 Prueba 02 - Recurso inexistente

Fecha / plataforma: 29/09/2026, 20:50; Android Emulator API 37.1.

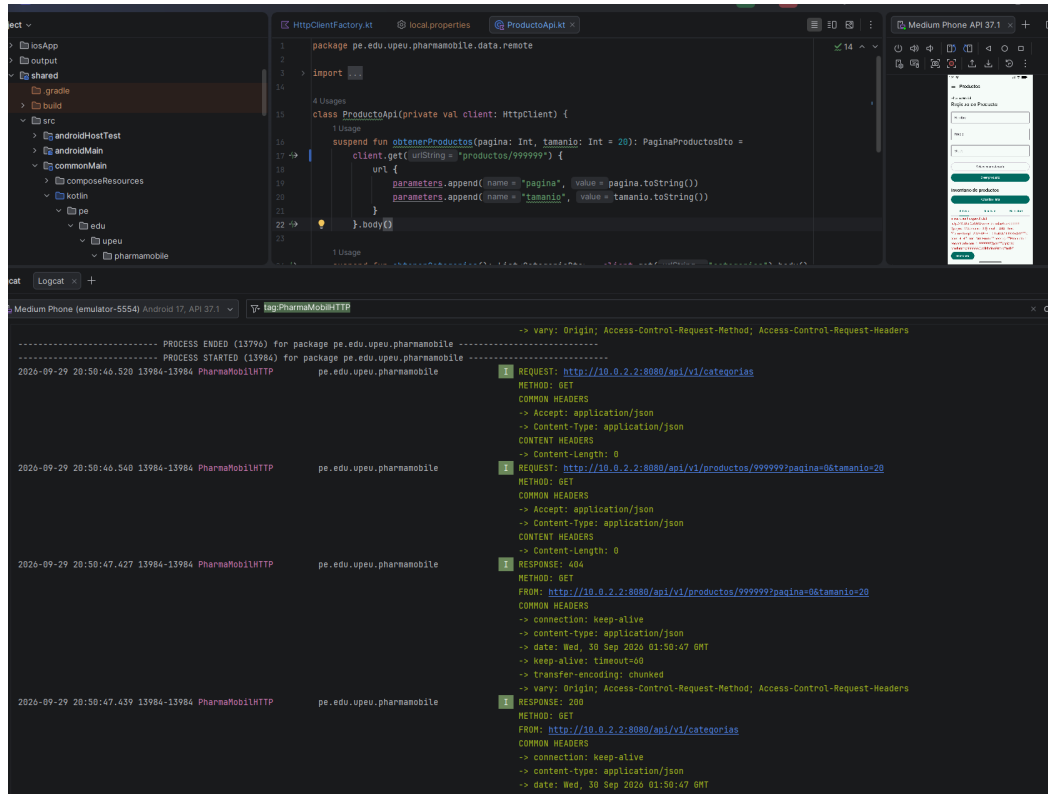
Pasos: Cambiar temporalmente `client.get("productos")` a `client.get("productos/999999")` en `ProductoApi`, ejecutar de nuevo la app y consultar la lista.

Esperado: HTTP 404, `ClientRequestException` capturada y mensaje controlado.

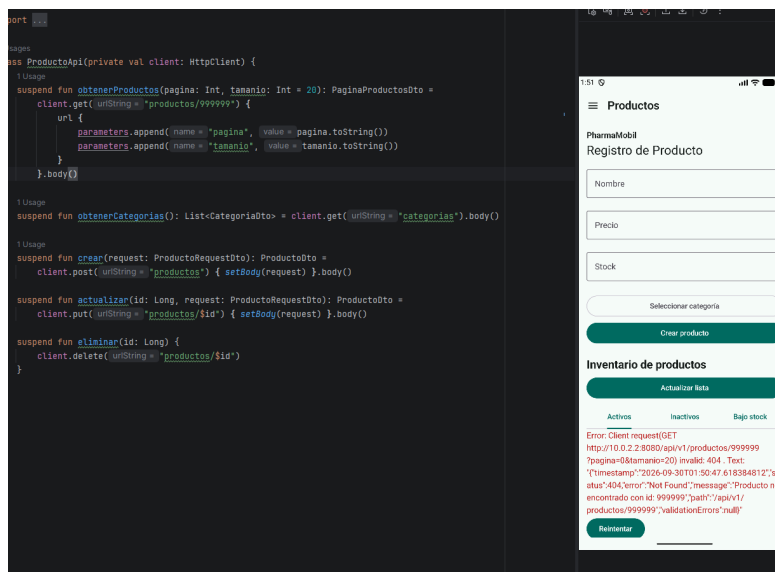
Observado: Logcat registra GET /productos/999999 a 404. La app no se cierra, pero muestra el texto completo de la excepción y del JSON del backend; el mensaje no está simplificado.

Qué ve el usuario: Error técnico largo con 404 y «Producto no encontrado».

Conclusión: Manejo sin cierre correcto; presentación del error mejorable.



Evidencia 02A. URL inexistente y respuesta 404.



Evidencia 02B. Mensaje observado en la app.

3.3 Prueba 03 - Sin conexión y recuperación

Fecha / plataforma: 29/09/2026, 13:45 (hora mostrada por el emulador); Android Emulator API 37.1.

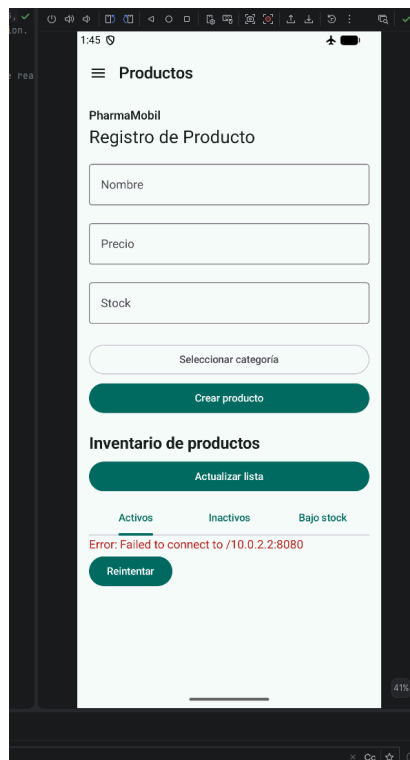
Pasos: Activar modo avión, pulsar Actualizar lista; desactivar modo avión y pulsar Reintentar.

Esperado: Fallo de entrada/salida capturado, mensaje y recuperación tras reintento.

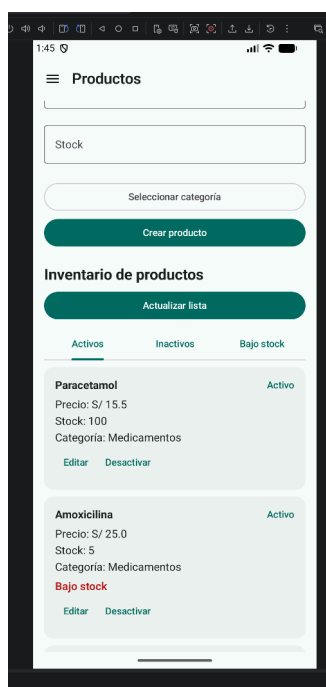
Observado: La app muestra «Failed to connect to /10.0.2.2:8080» sin cerrarse. Tras reintentar, vuelve a listar los productos.

Qué ve el usuario: Texto técnico de conexión y botón Reintentar; luego listado normal.

Conclusión: Recuperación funcional; conviene simplificar el texto de error.



Evidencia 03A. Modo avión y error de conexión.



Evidencia 03B. Productos cargados tras recuperar conexión.

3.4 Prueba 04 - Tiempo de espera agotado

Fecha / plataforma: 29/09/2026, 20:53; Android Emulator API 37.1.

Pasos: Cambiar temporalmente requestTimeoutMillis de 15_000 a 1 en HttpClientFactory; ejecutar de nuevo y actualizar la lista.

Esperado: Timeout manejado sin bloqueo de interfaz.

Observado: Logcat registra «Request timeout has expired» para categorías y productos, con request_timeout=1 ms. El registro menciona CancellationException; la app muestra el vencimiento y sigue operativa.

Qué ve el usuario: Error de timeout y opción de reintentar.

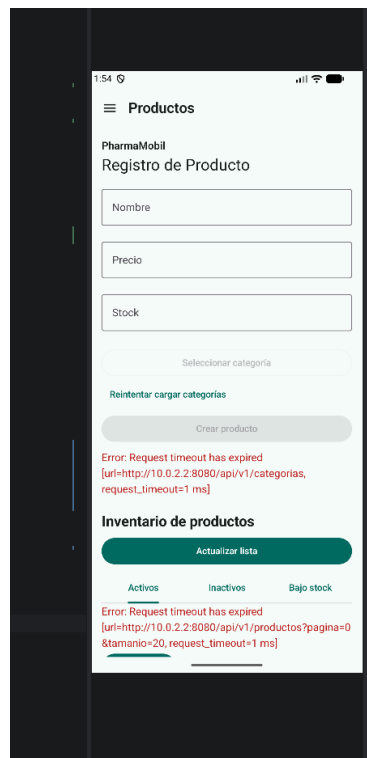
Conclusión: Timeout reproducido; no se afirma HttpRequestTimeoutException porque la traza observada nombra CancellationException.

```

METHOD: GET
FROM: http://10.0.2.2:8080/api/v1/categorias
COMMON HEADERS
-> connection: keep-alive
-> content-type: application/json
-> date: Wed, 30 Sep 2026 01:50:47 GMT
-> keep-alive: timeout=60
-> transfer-encoding: chunked
-> vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
1 REQUEST: http://10.0.2.2:8080/api/v1/categorias
METHOD: GET
COMMON HEADERS
-> Accept: application/json
-> Content-Type: application/json
CONTENT HEADERS
-> Content-Length: 0
1 REQUEST: http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20
METHOD: GET
COMMON HEADERS
-> Accept: application/json
-> Content-Type: application/json
CONTENT HEADERS
-> Content-Length: 0
1 REQUEST http://10.0.2.2:8080/api/v1/categorias failed with exception: java.util.concurrent.CancellationException: Request timeout has expired [url=http://10.0.2.2:8080/api/v1/categorias, request_timeout=1 ms]
1 REQUEST http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20 failed with exception: java.util.concurrent.CancellationException: Request timeout has expired [url=http://10.0.2.2:8080/api/v1/productos?pagina=0&tamano=20, request_timeout=1 ms]

```

Evidencia 04A. Timeout de 1 ms en Logcat.



Evidencia 04B. Interfaz abierta con error de timeout.

3.5 Prueba 05 - Campo JSON desconocido

Fecha / plataforma: 29/09/2026, 20:59; Android Emulator API 37.1.

Pasos: Cambiar temporalmente ignoreUnknownKeys de true a false, ejecutar de nuevo y actualizar la lista. Después restaurar true.

Esperado: Con false, excepción de serialización; con true, lectura normal.

Observado: El backend responde HTTP 200, pero Ktor registra JsonConvertException: categoria contiene descripcion y producto contiene fechaCreacion, campos no declarados en los DTO. Con true se obtuvo la lista en la prueba 01.

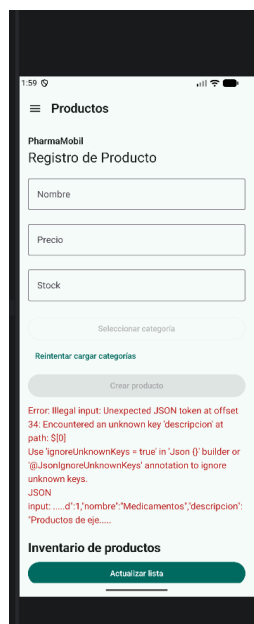
Qué ve el usuario: Error técnico «Encountered an unknown key»; la app no se cierra.

Conclusión: Se verificó la necesidad de ignorar campos adicionales en este contrato.

```

1 REQUEST http://10.0.2.2:8080/api/v1/categorias failed with exception: java.util.concurrent.CancellationException: Request timeout has expired [url=http://10.0.2.2:8080/api/v1/categorias, request_timeout=1 ms]
1 REQUEST http://10.0.2.2:8080/api/v1/productos?pagina=0&items=20 failed with exception: java.util.concurrent.CancellationException: Request timeout has expired [url=http://10.0.2.2:8080/api/v1/productos?pagina=0&items=20, request_timeout=1 ms]
1 REQUEST http://10.0.2.2:8080/api/v1/categorias
METHOD: GET
COMMON HEADERS
-> Accept: application/json
-> Content-Type: application/json
CONTENT HEADERS
-> Content-Length: 0
1 REQUEST http://10.0.2.2:8080/api/v1/productos?pagina=0&items=20
METHOD: GET
COMMON HEADERS
-> Accept: application/json
-> Content-Type: application/json
CONTENT HEADERS
-> Content-Length: 0
1 RESPONSE: 200
METHOD: GET
FROM: http://10.0.2.2:8080/api/v1/categorias
COMMON HEADERS
-> connection: keep-alive
-> content-type: application/json
-> date: Wed, 30 Sep 2026 01:59:42 GMT
-> keep-alive: timeout=60
-> transfer-encoding: chunked
-> vary: origin, Access-Control-Request-Method, Access-Control-Request-Headers
1 RESPONSE http://10.0.2.2:8080/api/v1/categorias failed with exception: io.ktor.serialization.JsonConvertException: Illegal input: Unexpected JSON token at offset 34: Encountered an unknown key 'descripcion' at path: $[0]
Use 'ignoreUnknownKeys = true' in 'Json ()' builder or '@JsonIgnoreUnknownKeys' annotation to ignore unknown keys.
JSON input: ....d"1,"nombre":"Medicamentos","descripcion":"Productos de eje....
1 RESPONSE: 200
METHOD: GET
FROM: http://10.0.2.2:8080/api/v1/productos?pagina=0&items=20
COMMON HEADERS
-> connection: keep-alive
-> content-type: application/json
-> date: Wed, 30 Sep 2026 01:59:42 GMT
-> keep-alive: timeout=60
-> transfer-encoding: chunked
-> vary: origin, Access-Control-Request-Method, Access-Control-Request-Headers
1 RESPONSE http://10.0.2.2:8080/api/v1/productos?pagina=0&items=20 failed with exception: io.ktor.serialization.JsonConvertException: Illegal input: Unexpected JSON token at offset 135: Encountered an unknown key 'fechaCreacion' at path: $[0]
Use 'ignoreUnknownKeys = true' in 'Json ()' builder or '@JsonIgnoreUnknownKeys' annotation to ignore unknown keys.
JSON input: ....egoriaNombre":"Medicamentos","fechaCreacion":"2026-09-29T20:....
  
```

Evidencia 05A. HTTP 200 seguido del fallo de deserialización.



Evidencia 05B. Error visible para el usuario.

4. Repositorio, configuración final y conclusiones

Configuración final verificada en el código: `client.get("productos")`, `requestTimeoutMillis = 15_000` e `ignoreUnknownKeys = true`. README.md contiene la sección «Conectividad REST» y las instrucciones de ejecución con Docker.

Commit de código y evidencias: 375b4c1

Backend: <https://github.com/Akatosh4019/BackendPharmobile>

Los cinco escenarios fueron ejecutados. La app maneja las fallas sin cerrarse, aunque la presentación de 404, conexión, timeout y JSON aún muestra mensajes técnicos. Este resultado se declara como oportunidad de mejora y no como cumplimiento completo del criterio de mensajes controlados.

La captura de Logcat registra método, URL, cabeceras y estado; la configuración usa `LogLevel.HEADERS`, por lo que no imprime cuerpos JSON. El fragmento JSON de la sección 2 fue obtenido directamente del backend local.

Referencia de la actividad

Universidad Peruana Unión. Actividad Autónoma N.º 07 - Documentación de endpoints, DTO y pruebas de conexión, sesión 7, Desarrollo de Aplicaciones Móviles, 2026-2.